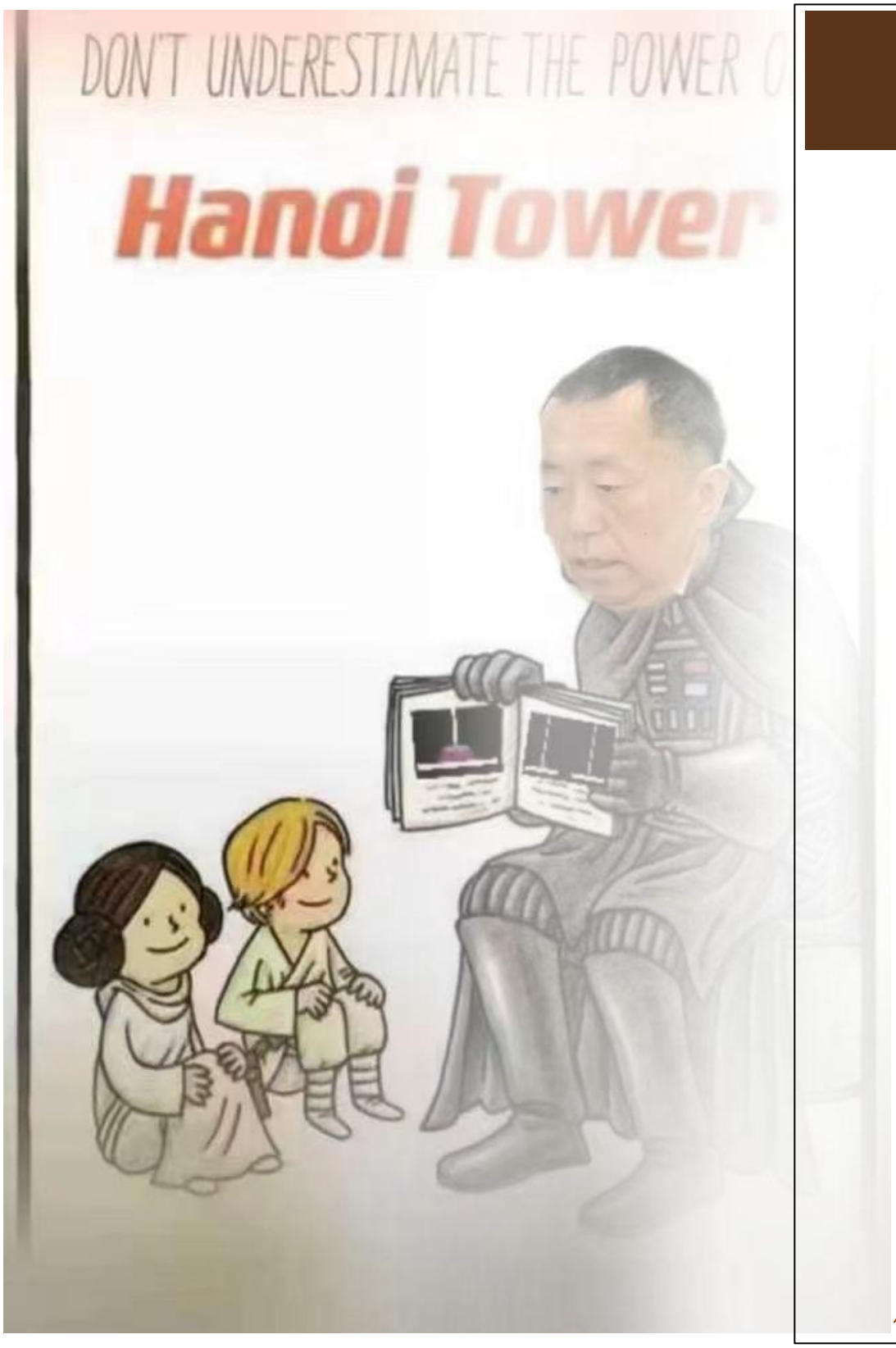


汉诺塔综合演示报告

信12 2350222 盛曦

完成日期：2024.5.13

1. 题目（黑体，小三号，段前段后间距各1行）

1.1.次级小标题（如果有，宋体，四号，加粗，段前段后间距各1行）

1.1.1. 三级小标题（如果有，宋体，小四，加粗，段前段后间距各1行）

将以往做过的汉诺塔题目集成在一个程序中，并额外加入了伪图形的显示。需要做到：
按步展示
显示内部的横向、纵向数组
绘制圆柱、盘子
呈现盘子的移动过程、根据用户指令进行盘子移动等。
整个程序建了多个源文件和头文件，函数也有规范要求，需要注意各区域代码的协调

2. 整体设计思路

程序中，有2个头文件，4个源文件。
头文件：
cmd_console_tools.h：伪图形所需要用到的函数声明、宏定义等
hanoi.h：实现各个汉诺塔solutions所需要的函数声明、宏定义等
源文件：
cmd_console_tools.cpp：伪图形需要的函数的具体内容
hanoi_main.cpp:程序主体的不断循环、根据菜单返回值调用solutions中的不同函数
hanoi_menu.cpp：菜单函数
hanoi_multiple_solutions.cpp:存放用于实现所有的具体到菜单中每一项功能的函数
可见，设计思路主要集中体现于solutions源文件中的各项函数。在这些函数中，大致可以分为几块：
一，输入函数，读入键盘输入的汉诺塔参数/游戏时读入用户的指令：需要特别注意输入错误处理，每次开始时得清零全局变量
二，输出文字版汉诺塔的函数，比如输出横向、纵向数组：需要注意输出的位置会根据菜单选项的不同而改变
三，伪图形汉诺塔的函数，包括画柱子盘子、移动盘子
四，其他杂功能，如输出提示词、延时等
五，递归函数：集成上述所有的函数，实现一整个汉诺塔移动的过程
六，主函数：调用递归函数，进行一些细节处理，供main源文件中的main函数调用
这些函数的形参常是起始和目标柱、菜单里选择的模式。
整个程序使用一个全局二维数组存放三个柱子上的盘子plate[3][11]，一个全局一维数组存放柱子上最顶层的盘子top[3]，以便在繁杂众多的函数中传递盘子的移动情况

3. 主要功能的实现

3.1menu源文件：

3.1.1menu函数：返回char型的mode，无形参。清屏并打印所有菜单选项，使用_getch读取用户选择的模式，输入错误则不回显并重输，直到正确。返回用户选择的模式mode

```
char mode = _getch();
while (mode != '1' && mode != '2' && mode != '3' && mode != '4' && mode != '5' &&
mode != '6' && mode != '7' && mode != '8' && mode != '9' && mode != '0')
{
mode = _getch();
}
return mode;
```

3.2main源文件

3.2.1main函数：循环进行以下操作（除非mode==0）:调用menu，根据返回的mode，调用不同的solutions中的主函数（上文第六类）或别的函数,在函数执行完毕（一个模式下的所有操作完成）后，输出让用户按回车继续的提示词（注意输出位置根据mode不同而不同，光标状态需改变），输入错误处理

3.3solutions源文件

3.3.1第一类输入函数：无返回，形参为指针变量。

放在第六类函数中的开头，处理多个参数（起始柱、目标柱、层数等）的输入，因为要返回多个值，所以用指针变量做形参。

输入错误处理规则为无论输入什么都要清空缓冲区，以免影响后面的输入。

除了输入参数外，我也在此函数中加入了全局变量清零、根据输入值给全局变量重新赋值的操作，因为这些同输入参数一样，都是在每个第六类函数中重复进行的。

特别地，给菜单9游戏模式中用户的输入写了一个input9函数，这部分的输入错误处理非常复杂（对我来说），具体操作也混杂在输入处理里了。后面第四板块再讲

以层数n的输入为例展示循环：

```
while (1)
{
    cout << "请输入汉诺塔的层数(1-10)" << endl;
    cin >> *n;
    cin.clear();
```

```
    cin.ignore(1410065407, '\n');//不论输入什么都要清空
    if (*n >= 1 && *n <= 10 && cin.good() == 1)
        break;
}
```

3.3.2第二类输出内部数组函数：所有print+plate、line、vertical、tower。无返回，形参为mode或无形参。

Print plate：打印横向数组，用的是plate[][]，先改变第一个[]内的值（哪个柱子）再改变第二个（柱子的第几层）

Print line：进行是否要print的判断，据此调用print plate。实际上可以和print plate合并。

Print vertical：打印纵向数组，与print plate类似不再赘述，每打印一个盘子要改变纵坐标

Print tower：打印文字版汉诺塔的底部+调用print vertical打印文字版汉诺塔的竖直串
值得注意的是，在这些函数中我根据形参mode的值来给变量space赋值——space变量决定了打印这些东西的起始纵坐标位置。若mode为8或9，则起始纵坐标要加一个恒定值，打印整体下移，给伪图形预留空间

Space的用途：

```
int space = ((mode == '8' || mode == '9') ? SPACE : 0);//SPACE为宏定义的下移距离
```

以print vertical为例展示循环：

```
for (int j = 0; j < 3; j++)//先变柱子
{
    for (int i = 0; i < top[j]; i++)//再变盘子
    {
        cct_gotoxy(12 + j * 3, 15 - i - 1 + space);
        cout << plate[j][i];
    }
}
```

3.3.3第三类伪图形函数：draw+poles， plates;move。无返回，形参为src，dst，mode

Draw poles:利用循环，先打印3个底座，再一层一层打印3个柱子。光标状态为不显示。菜单5调用

Draw plates：在起始柱上画n层盘子，同样是利用循环（不断改变plate[][]的第二个[]内的值，即改变盘子的层数），注意宽度为盘子编号*2+1，这样规则整齐；颜色为盘子编号

Move：移动盘子。先定义盘子的宽度，xy坐标、颜色，然后移动，先打印移动后的位置，再覆盖移动前的位置。移动分为3步：竖直上移：循环直至y到达指定值；平移：循环直至x到达指定值；竖直下移。注意：1，因为盘子是有宽度的，每次调用showch时的起始位置实际上不是x，而是X - (w - 1) / 2；2，竖直移动时还要格外重新打印柱子；3，竖直下移时在终点不用覆盖上一个；4，定义步长steplen来决定打印的位置，是否覆盖的条件，会更清楚简单。注意延时的设置会根据mode不同而不同

以竖直下移为例展示：

```
steplen = 1;//步长为1
for (; Y != BASE_Y - top[dst - 'A'] - 1; Y += steplen)
{
    cct_showch(X - (w - 1) / 2, Y + steplen, ' ', color, COLOR_WHITE, w);//打印下一次移动
    if (Y != BASE_Y - top[dst - 'A'])//终点处不用覆盖上一个
    {
        cct_showch(X - (w - 1) / 2, Y, ' ', COLOR_BLACK, COLOR_WHITE, w);//覆盖上一个盘子
        if (Y == 3)
        {

            cct_showch(X, Y, ' ', COLOR_BLACK, COLOR_WHITE, 1);//最高处用黑色覆盖上一次
        }
        else
        {
            cct_showch(X, Y, ' ', COLOR_HYELLOW, COLOR_WHITE, 1);//其他位置用黄色覆盖，重新画柱子
        }
    }
    Sleep(400 - 50 * delay);
}
```

3.3.4第四类杂功能函数：pause、change top、prompt

Pause：无返回无形参，根据delay的值进行延时

Change top：盘子移动导致的全局数组改变。将src的最顶层赋值给dst

```
plate[dst - 65][top[dst - 65]++] = plate[src - 65][--top[src - 65]];
plate[src - 65][top[src - 65]] = 0;
```

写这个只有两行的函数是因为历史遗留原因。我在先前的作业中先改变plate和top数组值再进行文字版汉诺塔的打印，导致调用move时数组的状态已经是移动后的了。为了让move不受其影响，得在move的开头把dst的最顶层赋给src（相当于逆转了这次移动），然后move，然后在src的最顶层赋给dst（补回先前的逆转）

prompt：输出提示词，根据mode不同决定是否要gotoxy，是否要输出第几步

3.3.5第五类递归函数：无返回，形参为n,src,tmp,dst,mode

Hanoi recur：主要功能：1，递归，汉诺塔问题的解决方法；调用change top以改变全局数组的值体现盘子的移动，调用prompt、pause，根据mode的值调用print line（打印横向数组）、print tower（打印纵向数组），move（打印伪图形移动）

```
if (n == 0)
    return;//移动完毕结束递归
hanoi_recur(n - 1, src, dst, tmp,mode);//递归
change_top(src, dst);
prompt(mode,src,dst);
if (mode != '1'&& mode != '2') {
    print_line(mode);
    if (mode != '3') {
        print_tower(mode);
        if (mode == '8')
            move(src, dst);
        pause();
    }
}
hanoi_recur(n - 1, tmp, src, dst,mode);//递归
```

正好15行

3.3.6第六类主函数

Hanoi123489：共菜单123489使用，共性是需要进行一系列打印（横向纵向数组，伪图形）、调用递归函数（除9）来呈现汉诺塔的移动过程。步骤为：

输入：调用输入函数

根据mode的不同来输出不同的提示词

做该模式开头的前置准备：根据mode的不同来决定：gotoxy到哪里；输出什么东西，即调用哪些第二、三类函数；怎么延时；其他的细节比如光标状态，space的大小等

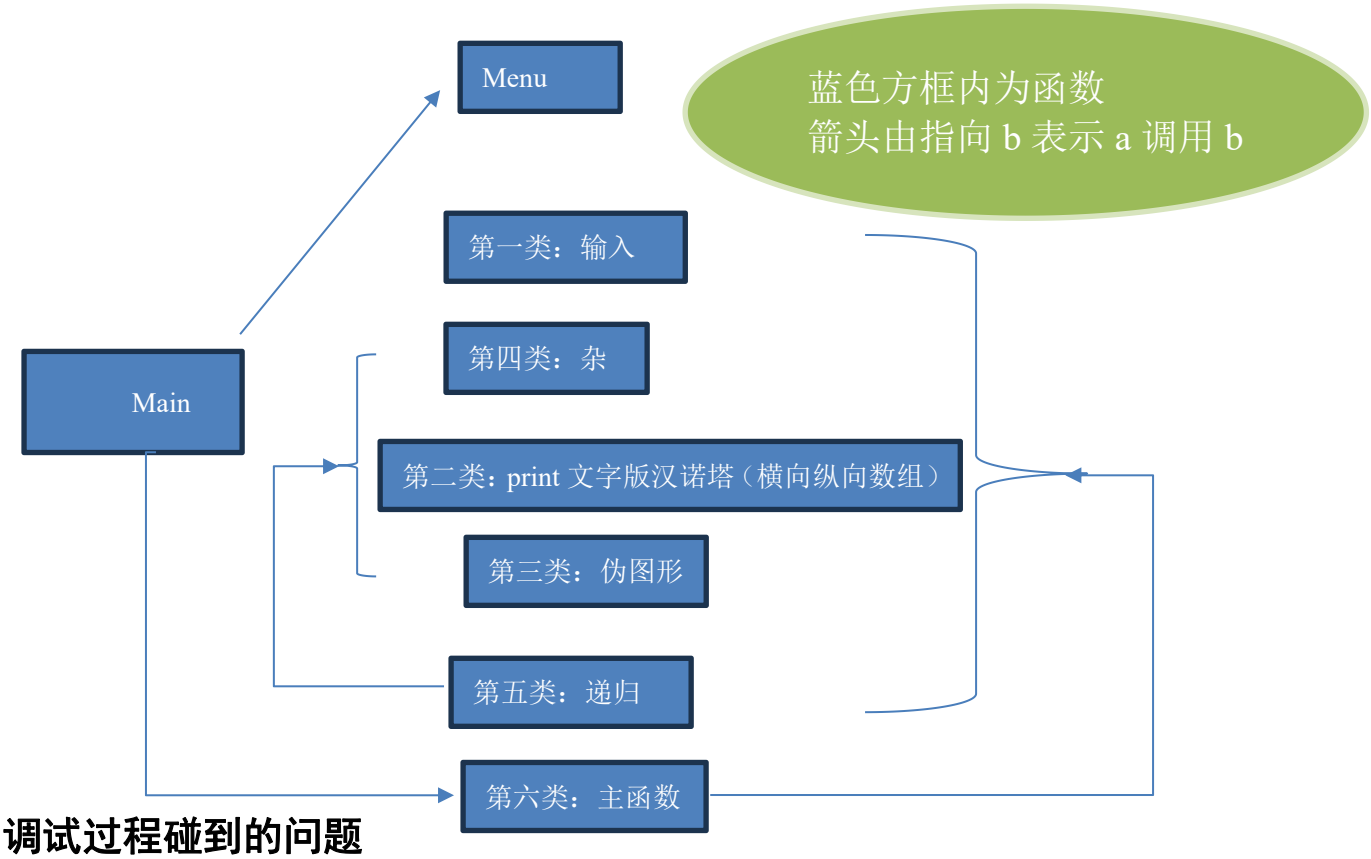
调用递归函数（12348）调用input9（9）

```
if (mode == '1'|| mode=='2')
{
    cout << "移动步骤为:" << endl;
}
if (mode == '4'||mode=='8'||mode=='9')
{
    int space= ((mode == '8' || mode == '9') ? SPACE : 0);
    cct_cls();
    cout << "从 " << src << " 移动到 " << dst << "，共 " << n << " 层";
    if (mode != 9)
    {
        cout <<"，延时设置为 " << delay << "。";
    }
    cct_gotoxy(20, 18+space);
    cout << "初始:" << setw(16) << " ";
    cct_setcursor(CURSOR_INVISIBLE);
    print_line(mode);
    print_tower(mode);
    if (mode == '8'||mode=='9')
    {
        draw_poles();
        draw_plates(src);
    }
    if (mode != '9')
    {
        pause();
    }
    else
    {
        input9(src,tmp);
```

```
    }
}
if (mode != '9')
{
    hanoi_recur(n, src, tmp, dst, mode);
}
```

Draw Hanoi plates：为了菜单6特地写的函数，负责调用输入函数和draw plates，起始柱上画一串盘子

以下为大致的各个函数相互作用图：



4. 调试过程碰到的问题

4.1以往各个作业的合并&历史遗留问题

我一开始阅读得不够认真，没有发现递归函数必须只能用一个，所以只是把以往作业的函数全拷过来了。在完成了整个程序的撰写和调试以后才发现这个问题，又要重新把这些乱七八糟的函数整理合并，合并完了还得重新调试一遍，非常麻烦。

合并的时候，有几个困难：

第一， 各个条件语句之间的逻辑必须清楚（一般是对mode值判断的条件）

第二， Space我本来是作为第二类函数的默认参数的，现在合并后几乎所有函数的形参都得改变

了，space如何处理也要改变，很麻烦

第三， 历史遗留问题。我在先前的作业中先改变plate和top数组值再进行文字版汉诺塔的打印，导致调用move时数组的状态已经是移动后的了。就出问题了。

解决方法：要解决前两个问题，就是列草稿理清逻辑，不断调试发现自己的错误之处，有点耗时间但不难;要解决第三个问题，得在move的开头把dst的最顶层赋给src（相当于逆转了这次移动），然后move，然后在src的最顶层赋给dst（补回先前的逆转）。这个解决方法其实并不好，但个人认为要重写一遍第二类函数的代价更大

4.2Input9（菜单9用户输入盘子移动指令）的输入错误处理

在用老师的demo时，我发现这部分的输入错误及对应处理有这么几种：

1， 输入两个合规字符但原柱为空：输出提示语并清空重新输入

2， 输入两个合规字符但大盘压小盘：输出提示语并清空重新输入

3， 输入了超过2个字符但没超过20个：按回车后清空重新输入，无其他反应

4， 输入了超过20个字符：在输入第20个的瞬间直接清空重新输入

5， 输入有回显，除了空格

据此我判断输进去的那个字符数组大小应该是20，输入方式应当是_getch（因为空格无回显），其他的一步步处理就是细节了。我写了很久，就是无法完全符合上面五条要求，问题包括但不限于：不小心_getch了上一次输入留下的东西（一般在输入的东西超过20个字符时发生），为了判断getchar的大小导致多写了getchar，要按两次回车才有反应（老问题了），清空缓冲区的方法错误（不能用while(getchar()!=’\n’，因为有可能上一次输入的回车已经被吸收了，这种方式清空缓冲区会导致你必须按一次回车，实际上这种情况用不着清空）

解决这些问题的方法当然就是不停地调试、发现问题、改正、再调试。在使用调试工具的时候我找不到程序在等待输入时执行的是哪一行代码，这给我带来了一些麻烦。

最后还是成功写出了一个很长的input9达到了要求（应该吧）。

4.3细节问题的查找

在调试程序的时候，常发现一些细节问题，包括但不限于：光标的显示状态、字体颜色设置、细节输入错误处理等。

要解决这个问题，可以使用调试工具，现在自己认为会有问题的地方设置断点，再调试、监视变量等等，有效地发现问题结症所在。

5. 心得体会

5.1 完成本次作业得到的一些心得体会，经验教训

一定要先仔细阅读老师发的要求。。假如做完了再读再改会很麻烦

调试报告需加快进程，很多时候因为不会调试而觉得找错误很累

善用函数间的互相调用，这需要好好利用形参。这样可以减少函数个数，使代码更简洁，逻辑更清晰

5.2 在做一些较为复杂的程序时，是分为若干小题还是直接一道大题的形式更好？

分为若干小题更好。将大题目拆解，使得更容易下手去做，也更容易厘清各个板块之间的逻辑。刚做这个大作业时看到这么多菜单觉得很慌，但是实际上很多都是以往做过的组合在一起，实际上并没有那么难。

5.3 汉诺塔完成了若干小题，总结你在完成过程中是否考虑了前后小题的关联关系，是否能尽

可能做到后面小题有效利用前面小题已完成的代码，如何才能更好地重用代码？

有些时候我没有考虑清楚。比如我在变量和函数的命名、宏定义或const的使用时有所欠缺，导致我有时候甚至看自己以前写的代码都觉得费劲；一些以前写的函数中，本可以设置为变量的数却被我设置成了常量，虽然在当时这个值可能没有变，但是合并进大作业就变了，比如打印横向纵向数组的纵坐标位置。

在以后的编程中，我一定会多用宏定义和const，默认参数等，尽可能地考虑甲方会给出什么样的修改要求，哪些参数是可能改变的，这个函数以后可能被用在什么样的工程里。

5.4 已本次作业为例，说明如何才能更好地利用函数来编写复杂的程序

避免重复功能的函数。尽量将同样功能写成同一个函数，细节差异用参数体现

一个函数不宜太长，即不宜包含过多的功能。将功能拆分到不同函数，更方便修改维护，看起来也更清楚

多用宏定义和const，默认参数，当程序实现的功能改变/调试发现问题时，可能仅需修改这部分内容，会使代码的维护修改变得简单

必要时将函数们分到不同源文件中，再用头文件声明，看起来更清楚

6. 附件：源程序

Hanoi.h:

```
#define DS 32 //32为相邻两个底座/柱子之间的距离
#define BASE_Y 16 //底座的纵坐标
#define POLELEN 12//柱子的高度
#define BASELEN 23//底座的长度
#define SPACE 18//打印图形汉诺后，文字汉诺塔需要下移18个单位
char menu();
void hanoi123489(char mode);
void draw_poles();
void hanoi_draw_plates();
void hanoi_MoveOnePlate();
```

hanoi_multiple_solutions.cpp:

```
//第二类函数,这里就放 1 个了：
void print_vertical(char mode)
{
    int space = ((mode == '8' || mode == '9') ? SPACE : 0);
    for (int j = 0; j < 3; j++)
    {
        for (int i = 0; i < top[j]; i++)
        {
            cct_gotoxy(12 + j * 3, 15 - i - 1 + space);
            cout << plate[j][i];
        }
    }
}
//第三类函数，打印柱子太简单了略
void move(char src, char dst)
{
    change_top(dst, src);
    cct_setcursor(CURSOR_INVISIBLE);
    int steplen;
    int w = plate[src - 'A'][top[src - 'A'] - 1] * 2 + 1;//当前打印盘子的宽度
    int X = POLELEN + (src - 'A') * DS;//盘子的横坐标
    int Y = BASE_Y - top[src - 'A'];//当前盘子的纵坐标
    cct_gotoxy(X, Y);
    int color = plate[src - 'A'][top[src - 'A'] - 1];//当前盘子颜色

    //竖直上移和平移略
    //竖直下移
    steplen = 1;//步长为1
    for (; Y != BASE_Y - top[dst - 'A'] - 1; Y += steplen)
    {
        cct_showch(X - (w - 1) / 2, Y + steplen, ' ', color, COLOR_WHITE, w);//打印下一次移动
        if (Y != BASE_Y - top[dst - 'A'])//终点处不用覆盖上一个
        {
            cct_showch(X - (w - 1) / 2, Y, ' ', COLOR_BLACK, COLOR_WHITE, w);//覆盖上一个盘子
            if (Y == 3)
            {

                cct_showch(X, Y, ' ', COLOR_BLACK, COLOR_WHITE, 1);//最高处用黑色覆盖上一次
            }
            else
            {
                cct_showch(X, Y, ' ', COLOR_HYELLOW, COLOR_WHITE, 1);//其他位置用黄色覆盖，重新画柱子
            }
```

```
        }
        Sleep(400 - 50 * delay);
    }
    change_top(src, dst);
}
void draw_plates(char pole)//在pole柱子上画盘子。
{
    cct_setcursor(CURSOR_INVISIBLE);
    int i, k, w;
    const int X = POLELEN + (pole - 'A') * DS;//X为pole的x坐标
    for (i = 1; i <= top[pole - 'A']; i++)
    {
        w = plate[pole - 'A'][i - 1] * 2 + 1;//当前打印盘子的宽度
        for (k = X - (w - 1) / 2; k <= X + (w - 1) / 2; k++)
        {
            cct_showch(k, BASE_Y - i, ' ', plate[pole - 'A'][i - 1], COLOR_WHITE, 1);//盘子是几号，颜色就是
几号色
        }
        Sleep(22);
    }
}

//递归
void hanoi_recur(int n, char src, char tmp, char dst,char mode)
{
    if (n == 0)
        return;
    hanoi_recur(n - 1, src, dst, tmp,mode);
    change_top(src, dst);
    prompt(mode,src,dst);
    if (mode != '1'&& mode != '2') {
        print_line(mode);
        if (mode != '3') {
            print_tower(mode);
            if (mode == '8')
                move(src, dst);
            pause();
        }
    }
    hanoi_recur(n - 1, tmp, src, dst,mode);
}
//主要函数
void hanoi123489(char mode)
{
    //调用input略
    if (mode == '1'|| mode=='2')
    {
        cout << "移动步骤为:" << endl;
    }
    if (mode == '4'||mode=='8'||mode=='9')
    {
        int space= ((mode == '8' || mode == '9') ? SPACE : 0);
        cct_cls();
        cout << "从 " << src << " 移动到 " << dst << ", 共 " << n << " 层";
        if (mode != 9)
        {
            cout <<", 延时设置为 " << delay << "。";
```

```
        }
        cct_gotoxy(20, 18+space);
        cout << "初始:" << setw(16) << " ";
        cct_setcursor(CURSOR_INVISIBLE);
        print_line(mode);
        print_tower(mode);
        if (mode == '8'||mode=='9')
        {
            draw_poles();
            draw_plates(src);
        }
        if (mode != '9')
        {
            pause();
        }
        else
        {
            input9(src,tmp);
        }
    }
    if (mode != '9')
    {
        hanoi_recur(n, src, tmp, dst, mode);
    }
}
void input9(char src, char tmp)
{
    int i, j;
    char mode = '9';
    char cmd[20] = { '\0' };
    char m;
    cct_showch(0, 19 + SPACE, ' ', COLOR_BLACK, COLOR_WHITE, 0);//颜色回复默认
    cout << "请输入移动的柱号(命令形式：AC=A顶端的盘子移动到C，Q=退出) ：";//这一行文字长度为60
    while (1){
        cct_showch(60, 19 + SPACE, ' ', COLOR_BLACK, COLOR_WHITE, 20);//把上一次打出来的东西删了
        cct_gotoxy(60, 19 + SPACE);
        cct_setcursor(CURSOR_VISIBLE_NORMAL);
        for (i = 0; i < 20; i++){
            cmd[i] = '\0';
        }
        for (i = 0; i < 20; i++)
        {
            m = _getch();
            if (m != ' ')
            {
                cmd[i] = m;
                cout << m;
            }
            else
            {
                i--;
            }
            if (m == '\n' || m == '\r')
            {
                break;
            }
        }
```

```
        if (i == 20)
        {
            continue;
        }
        if (cmd[0] != 'q' && cmd[0] != 'Q')
        {
            if (cmd[3] != '\0')//输入超过3个(2个字母+回车），直接重输
            {
                continue;
            }
            for (j = 0; j < 2; j++)//输入了两个，把这两个小写化大写
            {
                if ('a' <= cmd[j] && cmd[j] <= 'c')
                {
                    cmd[j] += ('A' - 'a');
                }
            }
            if (cmd[0] >= 'A' && cmd[0] <= 'C' && cmd[1] >= 'A' && cmd[1] <= 'C' && cmd[1] != cmd[0])
            {
                if (top[cmd[0] - 'A'] == 0)
                {
                    cct_gotoxy(0, 20 + SPACE);
                    cout << "源柱为空！";
                    Sleep(1000);
                    cct_showch(0, 20 + SPACE, ' ', COLOR_BLACK, COLOR_WHITE, 30);
                    continue;
                }
                else
                {
                    if (top[cmd[1] - 'A'] != 0)
                    {
                        if (plate[cmd[0] - 'A'][top[cmd[0] - 'A'] - 1] > plate[cmd[1] - 'A'][top[cmd[1] - 'A']
- 1])
                        {
                            cct_gotoxy(0, 20 + SPACE);
                            cout << "大盘压小盘，非法移动！";
                            Sleep(1000);
                            cct_showch(0, 20 + SPACE, ' ', COLOR_BLACK, COLOR_WHITE, 30);
                            continue;
                        }
                    }
                }
                plate[cmd[1] - 'A'][top[cmd[1] - 'A']++] = plate[cmd[0] - 'A'][--top[cmd[0] - 'A']];
                plate[cmd[0] - 'A'][top[cmd[0] - 'A']] = 0;
                cct_showch(0, 38, ' ', COLOR_BLACK, COLOR_WHITE, 1);
                cct_gotoxy(20, 18 + SPACE);
                cout << "第" << setw(4) << ++num << " 步(" << setw(2) << plate[cmd[1] - 65][top[cmd[1] - 65]
- 1] << "): " << cmd[0] << "-->" << cmd[1] << " ";
                print_line(mode);
                print_tower(mode);
                move(cmd[0], cmd[1]);
                if (top[tmp - 'A'] == 0 && top[src - 'A'] == 0)
                {
                    cct_showch(0, 20 + SPACE, ' ', COLOR_BLACK, COLOR_WHITE, 0);//颜色回复默认
                    cout << "游戏终止！！！";
                    break;
                }
```

```
            }
            continue;
        }
        else//输了q
        {
            if (cmd[2] != '\0')//输入超过一个（1个字母+回车），直接重新输
            {
                continue;
            }
            cct_gotoxy(0, 20 + SPACE);
            cout << "游戏终止！！！";
            break;
        }
    }
}
```